

Z-MESH: uma proposta de arquitetura Zero Trust para Micro-MCPs efêmeros em runtimes de sandbox distribuídos

Uma abordagem arquitetural para o acesso seguro de agentes de IA a ferramentas e sistemas corporativos por meio de runtimes sob demanda.

Autor: Rodrigo Persiani Ciscom

Colaborador: Rogerio Alves de Macedo

Versão: 0.1

Data: 03 de junho de 2026

Tipo de documento: Position Paper / Proposta Arquitetural

Status: Versão inicial para discussão técnica, validação experimental e colaboração acadêmica

Sumário

Resumo	4
1. Introdução	5
2. Problema	7
2.1. O MCP como resposta ao problema de integração N×M.....	7
2.2. Funcionamento conceitual dos servidores MCP	9
2.3. O modelo atual baseado em servidores MCP persistentes	10
2.4. Riscos operacionais e de segurança dos MCPs persistentes	11
2.5. Topologia atual com múltiplos MCPs persistentes	12
2.6. Síntese do problema	13
3. Concepção e Fundamentação da Arquitetura Z-MESH.....	13
3.1. Plataformas de sandbox para agentes de IA e sua relação com o Z-MESH 15	
3.2. Centralizadores e orquestradores existentes	15
3.3. Diferencial arquitetural do Z-MESH	17
3.4. Hipótese da proposta.....	17
4. Visão geral da arquitetura Z-MESH	19
4.1. Camada de agentes e orquestradores.....	19
4.2. Concentrador Z-MESH	20
4.3. Motor de políticas, identidade e segredos	21
4.4. Runtimes de sandbox distribuídos	22
4.4.1. Microsegmentação e controle de tráfego dos runtimes	22
4.4.2. Mitigação do <i>cold start</i>	23
4.5. Micro-MCPs efêmeros.....	23
4.5.1. Injeção Just-in-Time de credenciais.....	24
4.6. Síntese da arquitetura.....	24
4.7. Por que o Z-MESH é uma arquitetura Zero Trust	25
4.8. Vantagens arquiteturais do modelo Z-MESH	25
5. Da sandbox isolada ao hub de governança de Micro-MCPs	26
6. Proposta de validação experimental	27
7. Limitações da proposta.....	28
8. Trabalhos futuros e expansão para arquiteturas AI-WAF	28
9. Contribuições esperadas	29
10. Conclusão	30
11. Referências	32

Resumo

A adoção de agentes de inteligência artificial em ambientes corporativos cria uma nova superfície de exposição, especialmente quando esses agentes passam a interagir com sistemas internos, bancos de dados, APIs, repositórios documentais e ferramentas operacionais. Embora o Model Context Protocol (MCP) ofereça um modelo padronizado para integração entre aplicações de IA e serviços externos, a manutenção de servidores MCP persistentes, amplos e distribuídos pela rede pode gerar riscos relevantes, como excesso de permissões, exposição de credenciais, aumento da superfície de ataque, movimentação lateral e uso indevido de ferramentas por agentes comprometidos ou manipulados por prompt injection.

Este trabalho propõe a arquitetura Z-MESH — Zero Trust Micro-MCP Ephemeral Sandbox Hub, uma camada complementar de governança Zero Trust voltada ao controle do ciclo de vida de capacidades MCP acionadas por agentes de IA. A proposta não busca substituir o MCP, frameworks agentic ou plataformas de sandbox, mas adicionar um plano de controle seguro capaz de mediar o acesso de agentes a sistemas corporativos por meio de Micro-MCPs efêmeros, especializados e executados sob demanda em runtimes de sandbox isolados.

Nesse modelo, agentes de IA não acessam diretamente sistemas internos nem se conectam a servidores MCP permanentemente expostos. Em vez disso, solicitam capacidades ao Concentrador Z-MESH, responsável por autenticar o agente, avaliar políticas, selecionar ou criar o Micro-MCP adequado, instanciá-lo em ambiente isolado, emitir credenciais temporárias, restringir o tráfego permitido, executar a ação autorizada, consolidar o resultado, registrar auditoria e destruir o runtime ao final da execução.

A proposta transforma o uso operacional de capacidades MCP, tradicionalmente associadas a conectores persistentes, em um modelo baseado em capacidades temporárias, especializadas, governadas e auditáveis, criadas apenas para atender a uma finalidade autorizada. Com isso, o Z-MESH busca reduzir a necessidade de manter múltiplos servidores MCP espalhados pela rede, diminuir a complexidade de proteção individual desses conectores, limitar a janela temporal de exploração e aplicar, de forma integrada, princípios de Zero Trust, privilégio mínimo, efemeridade, isolamento contextual, credenciais temporárias, controle de tráfego e auditoria integral ao consumo de ferramentas por agentes de IA.

Por se tratar de uma versão conceitual, o trabalho apresenta a arquitetura como proposta inicial para discussão técnica, validação experimental futura e evolução em laboratório controlado, com vistas a avaliar sua viabilidade funcional em cenários corporativos sensíveis.

Palavras-chave: Zero Trust; MCP; Model Context Protocol; agentes de IA; Micro-MCP; sandbox; sandbox runtime; orquestração; runtime efêmero; segurança de IA; DevSecAI; tráfego LLM.

1. Introdução

A evolução dos agentes de inteligência artificial amplia a capacidade de automação em ambientes corporativos, permitindo que modelos e orquestradores realizem consultas, executem tarefas, manipulem dados e acionem ferramentas externas com crescente autonomia. Essa evolução, porém, desloca parte relevante do risco para a camada de integração entre os agentes de IA e os sistemas corporativos, especialmente quando esses agentes passam a interagir com APIs internas, bancos de dados, repositórios documentais, ferramentas operacionais e serviços críticos. Nesse novo cenário, a superfície de ataque deixa de estar limitada ao modelo ou à aplicação que o consome e passa a incluir também os conectores, protocolos, credenciais, permissões e fluxos de execução utilizados para transformar a intenção do agente em ações reais sobre o ambiente corporativo.

Nas arquiteturas de referência que vêm orientando a atual padronização do mercado, servidores MCP podem ser implantados como conectores persistentes entre agentes de IA e recursos internos. Embora essa abordagem padronize e simplifique a integração, ela pode reproduzir problemas clássicos de segurança já observados em APIs¹, integrações legadas e contas técnicas², tais como falhas de autorização, permissões excessivas³, credenciais permanentes ou

¹ OWASP Top 10 API Security Risks – 2023 - OWASP API Security Top 10

² Privileged Identity Playbook - IDManagement

³ - AC-6 LEAST PRIVILEGE | NIST SP 800-53

embutidas⁴, baixa granularidade de controle de acesso, dificuldade de auditoria e ampliação da superfície de ataque⁵. Esses riscos são particularmente relevantes em ambientes nos quais conectores permanecem expostos de forma contínua, acumulam escopos de acesso amplos ou executam ações em nome de usuários, aplicações ou processos sem validação contextual suficiente.

Além disso, a distribuição de múltiplos servidores MCP pela rede corporativa aumenta a complexidade operacional de segurança, uma vez que cada conector passa a exigir proteção, monitoramento, hardening, atualização, controle de acesso, gestão de credenciais e auditoria próprios. Em ambientes de grande escala, esse modelo pode manter conectores permanentemente disponíveis, com níveis variados de maturidade de segurança, elevando o esforço de governança e ampliando a janela temporal disponível para exploração por atacantes, agentes comprometidos ou fluxos manipulados por *prompt injection*. Esse cenário reforça a necessidade de uma abordagem baseada em execução efêmera, na qual capacidades MCP sejam criadas apenas sob demanda, em ambiente isolado, com escopo mínimo e governança centralizada.

A proposta Z-MESH parte da premissa de que agentes de IA não devem possuir acesso direto, amplo ou persistente a recursos corporativos. Embora o Model Context Protocol (MCP) já padronize a comunicação entre aplicações de IA e serviços externos, o Z-MESH propõe uma camada adicional de governança baseada em execução efêmera, isolamento e controle contextual. Nesse modelo, Micro-MCPs específicos são criados apenas quando acionados por uma requisição previamente autorizada, dentro de runtimes de sandbox isolados e com escopo mínimo de atuação. Assim, a arquitetura deixa de tratar a sandbox apenas como mecanismo de contenção e passa a utilizá-la como ambiente controlado de execução para capacidades MCP, sob coordenação de um concentrador Zero Trust responsável por avaliar políticas, instanciar a ferramenta necessária, emitir credenciais temporárias, restringir o tráfego permitido, executar a ação autorizada, registrar auditoria e destruir o runtime ao final da tarefa.

Dessa forma, este documento apresenta o Z-MESH como uma proposta arquitetural complementar ao ecossistema MCP, voltada à redução da exposição de conectores persistentes e à aplicação de princípios de Zero Trust, privilégio mínimo, efemeridade e auditoria no acesso de agentes de IA a sistemas corporativos sensíveis.

⁴ [CISA Releases Guidance on Credential Risks Associated with Potential Legacy Oracle Cloud Compromise](#)

⁵ [MCP Security - OWASP Cheat Sheet Series](#)

2. Problema

2.1. O MCP como resposta ao problema de integração $N \times M$

O mercado tem convergido para a adoção do Model Context Protocol (MCP) como padrão emergente de comunicação e integração entre agentes de inteligência artificial e serviços externos. O MCP é um protocolo aberto que define como aplicações de IA podem interagir, de forma padronizada, com ferramentas, serviços, dados e workflows externos.

Em sua arquitetura básica, o MCP adota um modelo cliente-servidor. Aplicações de IA, também chamadas de hosts, utilizam clientes MCP para se conectar a servidores MCP, responsáveis por disponibilizar capacidades como execução de ferramentas, acesso a recursos, fornecimento de contexto e interações estruturadas.

O MCP foi criado pela Anthropic e disponibilizado como padrão aberto em novembro de 2024⁶, com o objetivo de mitigar e reduzir a fragmentação das integrações entre modelos de IA e sistemas externos. Desde então, o protocolo passou a ser incorporado ou referenciado por diferentes ecossistemas de IA incluindo plataformas e documentações de grandes fornecedores, consolidando-se como uma abordagem relevante para permitir que agentes de IA interajam com dados, APIs, aplicações e serviços corporativos.

Essa adoção é impulsionada principalmente pela capacidade do MCP de reduzir o problema de integrações ponto a ponto entre modelos e sistemas. Em um modelo tradicional, cada modelo de IA precisaria de conectores específicos para cada sistema interno, criando uma relação de complexidade $N \times M$. Com MCP, modelos e sistemas passam a se conectar por meio de uma camada comum de integração, reduzindo o esforço conceitual para $N + M$.

A transição de um modelo de acoplamento ponto a ponto ($N \times M$) para um modelo baseado em barramento ou protocolo comum ($N + M$) é um princípio consolidado na engenharia de software para redução de complexidade em sistemas acoplados⁷. Na prática essa abordagem reduz drasticamente a fricção de desenvolvimento no acoplamento de sistemas inteligentes a ambientes legados e operacionais.

Para fins de exemplificação, considera-se o seguinte cenário:

O Cenário Sem o MCP: Complexidade $N \times M$

⁶ <https://www.anthropic.com/news/model-context-protocol>

⁷ HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Boston: Addison-Wesley, 2003.

Imagine que você tem 3 (três) modelos de IA diferentes ($N = 3$, ex: GPT-4, Claude e Gemini) e quer que todos eles consigam acessar 4 (quatro) sistemas internos de uma empresa ($M = 4$, ex: Banco de Dados, Contratos, APIs Legadas e SIEM).

- I. Sem um protocolo padronizado, torna-se necessária a construção de um conector customizado para cada relação entre modelo e sistema correspondente:
 - a. O GPT-4 demandará uma lógica específica para o Banco de Dados, outra para o Sistema de Contratos e uma terceira para a API Legada.
 - b. O Claude exigirá o desenvolvimento de outra camada de integração para esses mesmos quatro sistemas.
 - c. O Gemini demandará o mesmo esforço de desenvolvimento dedicado.

Sob este fluxo, o ecossistema exigirá o desenvolvimento e a manutenção de $(3 \times 4) = 12$ (doze) integrações distintas. Caso a instituição adote mais 2 (dois) modelos e implemente mais 3 (três) sistemas, a volumetria de conectores cresce de forma multiplicativa. Esse cenário eleva significativamente o custo operacional, dificulta a manutenção e impede a reutilização de código.

Cenário **com o MCP**: Redução para $N + M$

O MCP atua como uma interface universal de conectividade para a Inteligência Artificial, de forma análoga ao padrão USB, que elimina a necessidade de conectores exclusivos para cada periférico, os sistemas e as LLMs passam a utilizar uma camada de comunicação unificada, segmentando-se em duas fronteiras bem definidas:

- I. MCP Clients (Modelos/Agentes): Consumidores que interpretam uma linguagem padronizada de chamadas.
- II. MCP Servers (Sistemas Provedores): Componentes que expõem dados e funcionalidades sob a mesma estrutura de comunicação.

Retornando ao cenário anterior com 3 (três) modelos (N) e 4 (quatro) sistemas (M) a dinâmica é simplificada:

- I. Garante-se que os 3 (três) modelos ofereçam suporte nativo ao protocolo MCP ($N = 3$).
- II. Envelopam-se os 4 (quatro) sistemas sob a especificação de servidores MCP ($M = 4$).

Dessa forma, o total de conexões necessárias reduz-se a $(3 + 4) = 7$ (sete) integrações.

A **Figura 1** demonstra o ganho de escalabilidade proporcionado pelo MCP em ambientes corporativos com múltiplos modelos de IA e diversos sistemas internos. No modelo tradicional, sem MCP, cada modelo precisa ser integrado diretamente a cada sistema, fazendo a complexidade crescer em lógica **N × M**. Já com MCP, modelos e sistemas passam a se conectar por meio de uma camada comum de integração, reduzindo o esforço para **N + M**. Em escala, essa diferença se torna expressiva: No exemplo com 10 (dez) modelos e 50 (cinquenta) sistemas, o número de integrações cairia de 500 (quinhentos) para 60 (sessenta), representando uma redução aproximada de 88%(oitenta e oito por cento) no esforço de integração, além de simplificar manutenção, governança, reutilização de conectores e evolução da arquitetura.



Figura 1 – Comparação de escalabilidade MCP

2.2. Funcionamento conceitual dos servidores MCP

Em termos estruturais, um servidor MCP é o componente que implementa o protocolo e fornece capacidades aos agentes de IA por meio de três primitivas principais:

- I. **Tools:** funções executáveis que o agente pode invocar dinamicamente, como consultas a bancos de dados, chamadas de API ou execução de ações em sistemas.

- II. **Resources:** conjuntos de dados textuais ou binários disponibilizados para leitura pelo agente, como arquivos de log, registros de auditoria, bases documentais ou contextos operacionais.
- III. **Prompts:** modelos de contexto ou interações estruturadas que orientam o raciocínio do agente durante a execução de uma tarefa.

Esses servidores atuam como conectores entre agentes e sistemas internos ou externos. Podem ser implantados localmente ou remotamente, sendo responsáveis por intermediar a comunicação com APIs, bancos de dados, aplicações legadas, serviços corporativos e ferramentas operacionais.

2.3. O modelo atual baseado em servidores MCP persistentes

Antes de apresentar a proposta Z-MESH, é necessário compreender o modelo predominante de integração entre agentes de IA e sistemas corporativos por meio de servidores MCP tradicionais. Nesse arranjo, os servidores MCP funcionam como conectores estáticos entre o agente e os recursos internos da organização, permanecendo disponíveis na rede para receber chamadas, expor ferramentas e intermediar o acesso a serviços de destino.

A **Figura 2** apresenta o modelo atual de execução de agentes baseado em servidores MCP persistentes. Nesse fluxo, a interação entre o agente de IA e os sistemas corporativos ocorre por meio de conectores mantidos continuamente ativos na infraestrutura de rede. O agente estabelece uma comunicação estruturada com um ou mais servidores MCP, que atuam como intermediários entre a lógica cognitiva do modelo e os serviços de destino da instituição, tais como APIs internas, bancos de dados, repositórios documentais e aplicações legadas.

Embora esse modelo padronize a camada de integração e viabilize o uso de agentes em larga escala, ele resulta na consolidação de conectores persistentes e, em muitos casos, de escopo amplo. Por operarem de forma contínua e concentrarem múltiplas permissões técnicas, essas instâncias estáticas podem ampliar a superfície de ataque da organização, dificultar a aplicação de controles granulares de privilégio mínimo e elevar o raio de impacto potencial em cenários de uso indevido, injeção semântica, comprometimento de credenciais ou falhas de autorização.

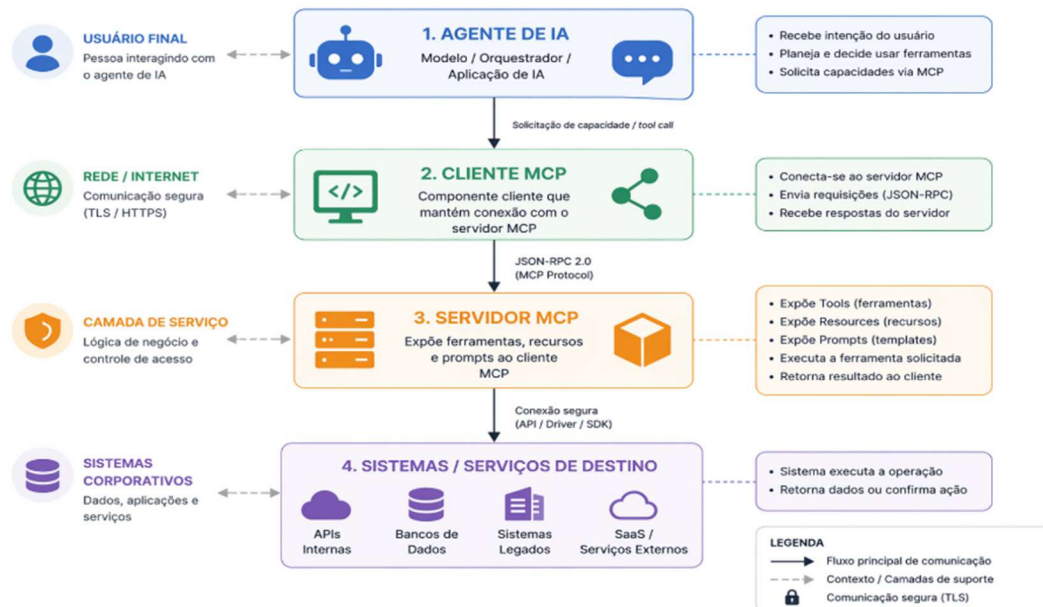


Figura 2 - Modelo atual de execução de agentes baseado em MCPs persistentes

2.4. Riscos operacionais e de segurança dos MCPs persistentes

Em ambientes corporativos atuais, a adoção do MCP pode ocorrer por meio da implantação de múltiplos servidores MCP persistentes, responsáveis por expor ferramentas, APIs, bancos de dados e serviços internos a agentes de IA. Esses componentes atuam como conectores especializados, frequentemente distribuídos pela rede corporativa para diferentes domínios de negócio, como gestão de contratos, dados financeiros, CMDB, SIEM, governança de cloud e aplicações legadas.

Nesse modelo, cada servidor MCP tende a manter conectividade contínua com os sistemas de destino e, em determinadas implementações, pode operar com credenciais previamente provisionadas, tokens de acesso estáticos ou contas técnicas com escopos excessivamente amplos. À medida que novos casos de uso emergem, novos servidores MCP são adicionados à infraestrutura, resultando em uma malha crescente de conectores persistentes com níveis heterogêneos de controle, governança e maturidade de segurança.

Esse cenário introduz riscos relevantes, tais como:

- Exposição prolongada e estática de servidores MCP na rede interna.
- Dependência de credenciais permanentes ou embutidas associadas a conectores ou contas técnicas.
- Ampliação do raio de impacto em cenários de comprometimento de um servidor MCP.
- Possibilidade de movimentação lateral a partir de conectores indevidamente segmentados.
- Aumento da complexidade de auditoria, limitação e rastreabilidade das ações executadas por agentes.
- Acionamento indevido de ferramentas críticas em razão de falhas de alinhamento, prompt injection ou erro de planejamento do agente;
- Consolidação de múltiplos privilégios em servidores MCP genéricos, resultando em conectores excessivamente permissivos.

Ou seja, na prática, o MCP⁸ atua como uma camada intermediária de abstração que permite que agentes de IA descubram, acessem e executem capacidades externas de forma padronizada, viabilizando arquiteturas mais dinâmicas, modulares e orientadas a agentes.

2.5. Topologia atual com múltiplos MCPs persistentes

Para ilustrar a arquitetura convencional adotada pelas organizações e evidenciar os riscos elencados, a **Figura 3**, apresenta uma topologia do cenário atual de múltiplos servidores MCP persistentes na rede corporativa.

Como se observa no diagrama, embora o perímetro inicial de acesso possa contar com componentes de governança, como AI Gateway ou API Gateway (Raia 2), a execução subsequente das capacidades (Raia 4) tende a depender de servidores MCP continuamente ativos e distribuídos pela rede. Esses conectores interagem com diferentes ambientes de serviços (Raia 5), como bancos de dados, aplicações internas, APIs, repositórios documentais e plataformas de gestão.

⁸ [MCP Gateway & Registry](#)

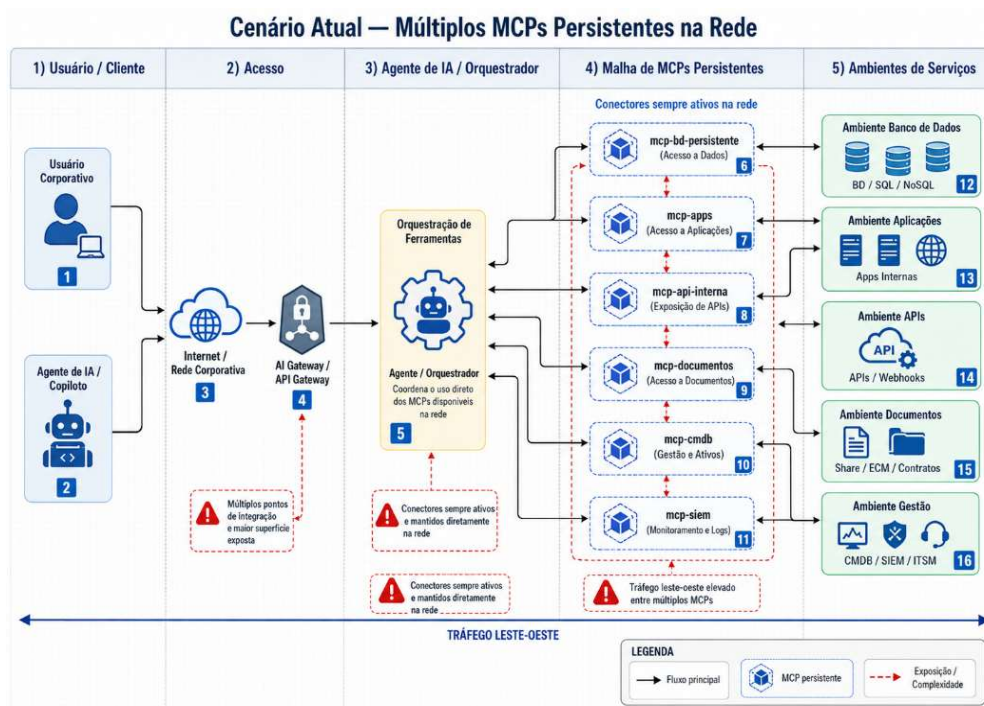


Figura 3 – Cenário Atual de múltiplos MCPs persistentes

Essa configuração gera aumento do tráfego Leste-Oeste, descentralização do controle de segurança e ampliação da superfície operacional de exposição. Cada servidor MCP passa a exigir *hardening*, atualização, monitoramento, controle de acesso, gestão de credenciais e auditoria próprios. Em caso de comprometimento de um conector estático, o atacante pode obter acesso direto ao segmento de rede, aos sistemas de destino ou às credenciais associadas àquele servidor.

2.6. Síntese do problema

O problema central não está no MCP como protocolo, mas no modelo operacional de implantação baseado em conectores persistentes, amplos e continuamente expostos. Embora o MCP represente um avanço relevante para padronizar integrações entre agentes de IA e sistemas corporativos, sua adoção em ambientes críticos exige uma camada adicional de governança, isolamento, controle de escopo, credenciais temporárias e auditoria.

Esse cenário reforça a necessidade de uma abordagem baseada em execução efêmera, na qual capacidades MCP sejam criadas apenas sob demanda, em ambientes isolados, com privilégio mínimo e governança centralizada. É nesse contexto que se insere a proposta Z-MESH.

3. Concepção e Fundamentação da Arquitetura Z-MESH

A formulação do Z-MESH emergiu da investigação sobre o comportamento dos fluxos associados a agentes de inteligência artificial em topologias corporativas, especialmente quando esses agentes passam a consumir ferramentas, APIs,

bancos de dados, repositórios documentais e serviços internos por meio de protocolos de integração como o MCP.

Durante o mapeamento de ameaças, observou-se que soluções tradicionais de sandbox, historicamente projetadas para emular sistemas operacionais isolados, executar binários suspeitos ou analisar artefatos maliciosos, apresentam efetividade limitada diante dos riscos específicos de aplicações generativas e agentes autônomos. Nesses cenários, o risco não se manifesta apenas como arquivo executável ou tráfego convencional, mas também como intenção semântica, *prompt injection*, abuso de ferramentas, vazamento de dados, uso indevido de credenciais e acionamento não autorizado de capacidades externas.

Em paralelo, o mercado passou a desenvolver plataformas voltadas à execução segura de código e ferramentas por agentes de IA. Soluções como E2B, infraestruturas baseadas em microVMs e ambientes especializados de sandbox demonstram a viabilidade de prover *runtimes* isolados e descartáveis para execução de scripts, comandos, manipulação de arquivos e uso de ferramentas em ambientes controlados.

O propósito principal dessas plataformas, contudo, está associado à disponibilização de um ambiente seguro para que agentes executem código, comandos ou ferramentas geradas dinamicamente. Elas funcionam como ambientes de computação isolados, úteis para reduzir o risco de execução direta no ambiente principal da aplicação. O Z-MESH parte dessa evolução, mas propõe um deslocamento de foco. Em vez de fornecer uma máquina isolada para execução ampla do agente, a arquitetura busca governar o ciclo de vida de capacidades MCP específicas, criadas sob demanda, com escopo mínimo, credenciais temporárias, controle de tráfego e destruição após a execução.

A partir desse panorama, emerge o seguinte questionamento: se as tecnologias atuais de virtualização leve e sandboxing conseguem instanciar ambientes efêmeros para execução de código, por que não aplicar a mesma lógica ao isolamento da infraestrutura de comunicação dos agentes, criando Micro-MCPs efêmeros, especializados e governados por política para mediar o acesso a sistemas corporativos sensíveis?

Essa hipótese fundamenta a reengenharia do modelo tradicional de integração por MCP. Na ausência de uma camada central de governança, servidores MCP dispersos pela rede corporativa podem operar como conectores persistentes, estáveis e continuamente expostos, tornando-se alvos previsíveis para exploração, uso indevido ou comprometimento. Ao transferir essas capacidades para runtimes de sandbox instanciados em tempo de execução, o Z-MESH reduz a janela de exposição dos conectores e limita sua existência ao tempo necessário para atender a uma requisição autorizada.

Adicionalmente, essa abordagem endereça o problema de servidores MCP excessivamente genéricos ou sobrecarregados de privilégios. Em vez de implantar um único conector amplo e permanentemente exposto, o Z-MESH atua como um hub dinâmico de capacidades. O tráfego oriundo do agente de IA é

recebido pelo concentrador, avaliado conforme identidade, finalidade, política e escopo, e direcionado à criação do Micro-MCP estritamente necessário para aquela tarefa.

Por exemplo, se o planejamento de um agente exigir uma sequência de ações, como consultar um saldo financeiro, extrair metadados de um contrato em PDF e verificar logs de auditoria, o Z-MESH não direciona o fluxo para conectores amplos e fixos. Em vez disso, provisiona, em tempo de execução, três ambientes independentes e isolados: um Micro-MCP com escopo exclusivo para leitura de banco de dados, um segundo runtime efêmero especializado em processamento documental e um terceiro voltado à consulta de eventos de segurança. Após o processamento das requisições e a devolução dos contextos estruturados ao agente, essas instâncias são encerradas, reduzindo a permanência de conectores ativos e retornando o ambiente ao estado de menor privilégio possível.

3.1. Plataformas de sandbox para agentes de IA e sua relação com o Z-MESH

Plataformas como E2B representam uma evolução relevante no ecossistema de agentes de IA, ao oferecerem ambientes isolados nos quais agentes podem executar código, comandos, ferramentas e integrações em um runtime controlado. Seu foco principal está em prover um ambiente operacional seguro para execução de código e ferramentas, com acesso a terminal, filesystem, rede e templates customizáveis dentro da sandbox.

O Z-MESH é complementar a esse tipo de plataforma. Enquanto soluções de sandbox agentic fornecem o plano de execução isolado, o Z-MESH propõe um plano de controle Zero Trust responsável por decidir quando uma capacidade pode ser acionada, qual Micro-MCP deve ser instanciado, qual credencial temporária será emitida, qual destino de rede poderá ser acessado, quais dados poderão retornar ao agente e quando o runtime deve ser destruído.

Dessa forma, o Z-MESH não substitui plataformas de sandbox para IA. Ele pode utilizá-las como camada de execução, adicionando governança, política, identidade, credenciais temporárias, egress restrito, auditoria e destruição pós-execução ao ciclo de vida de capacidades MCP.

3.2. Centralizadores e orquestradores existentes

A investigação também identificou frameworks capazes de centralizar ou orquestrar o uso de ferramentas por agentes de IA, incluindo ferramentas expostas por servidores MCP. No entanto, essas iniciativas estão predominantemente orientadas à experiência de desenvolvimento, à composição lógica de ferramentas e à coordenação de agentes, e não à governança de infraestrutura Zero Trust proposta pelo Z-MESH.

Microsoft Semantic Kernel

O Microsoft Semantic Kernel é um SDK/framework de orquestração de aplicações e agentes de IA que permite integrar modelos, prompts, funções,

plugins e ferramentas externas. No contexto do MCP, o Semantic Kernel permite adicionar ferramentas disponibilizadas por servidores MCP como plugins do Kernel, tornando essas capacidades acessíveis aos agentes e ao mecanismo de function calling.

Embora o Semantic Kernel facilite o consumo de ferramentas MCP e centralize sua exposição na camada de aplicação, ele não tem como finalidade principal governar o ciclo de vida Zero Trust dos servidores MCP subjacentes. O framework não transforma, por si só, cada ferramenta em um Micro-MCP efêmero, isolado por sandbox, com credencial temporária, egresso restrito, auditoria de runtime e destruição automática após a execução. Essas funções permanecem como responsabilidade da arquitetura de segurança implementada ao redor do framework.

LangChain e LangGraph

O ecossistema LangChain, em conjunto com LangGraph, oferece recursos avançados para construção de agentes, integração de ferramentas e modelagem de fluxos agênticos baseados em grafos de estado. No contexto do MCP, bibliotecas de integração permitem que agentes utilizem ferramentas definidas em um ou mais servidores MCP, convertendo essas ferramentas para o formato consumível por agentes LangChain e LangGraph.

O foco do ecossistema LangChain/LangGraph está na orquestração lógica do agente, no roteamento de ferramentas e na construção de workflows agênticos. O ciclo de vida seguro dos servidores MCP, sua implantação, isolamento de runtime, emissão de credenciais temporárias, controle de egresso e destruição pós-execução não constituem o objetivo central do framework. Assim, os servidores MCP consumidos continuam dependendo de uma arquitetura externa para garantir isolamento, privilégio mínimo e governança Zero Trust.

Microsoft Magentic-One / AutoGen

O Microsoft Magentic-One, no ecossistema AutoGen, é uma arquitetura multiagente voltada à resolução de tarefas complexas. Seu modelo utiliza um agente principal, denominado Orchestrator, responsável por planejar a tarefa, acompanhar o progresso, replanejar quando necessário e delegar subtarefas a agentes especializados, como WebSurfer, FileSurfer, Coder e ComputerTerminal.

O Magentic-One deve ser compreendido como uma arquitetura de orquestração multiagente, não como um concentrador nativo de MCPs ou como uma camada Zero Trust de gerenciamento de servidores MCP. Embora fragmente a execução entre agentes especializados, ele não substitui uma arquitetura dedicada à governança segura de ferramentas MCP, com Micro-MCPs efêmeros, credenciais JIT, controle de egresso, isolamento de runtime e destruição pós-execução.

3.3. Diferencial arquitetural do Z-MESH

A análise comparativa evidencia a lacuna que o Z-MESH busca endereçar. Enquanto frameworks como Semantic Kernel, LangChain, LangGraph e Magentic-One organizam a disponibilidade lógica de ferramentas para agentes, o Z-MESH atua sobre o ciclo de vida seguro das capacidades MCP na infraestrutura.

O diferencial do Z-MESH está em não gerenciar apenas quais ferramentas estão disponíveis, mas também como, onde, com qual escopo, por quanto tempo e sob quais políticas elas podem existir. Ao unificar centralização lógica, execução efêmera, isolamento de runtime, credenciais temporárias, egresso restrito e auditoria, o Z-MESH propõe uma camada complementar de segurança para ambientes corporativos críticos.

3.4. Hipótese da proposta

Com o propósito de mitigar vulnerabilidades associadas a conectores estáveis e solucionar o desafio de governança na camada de integração de agentes de IA, este trabalho propõe o **Z-MESH — Zero Trust Micro-MCP Ephemeral Sandbox Hub**. O Z-MESH é concebido como um ecossistema de orquestração e um plano de controle seguro, projetado para gerenciar o ciclo de vida e a execução de capacidades MCP sob uma abordagem defensiva e de privilégio mínimo.

A proposta parte do princípio de que nenhum componente de software, seja agente autônomo, LLM, aplicação ou conector de sistema, deve possuir confiança implícita ou persistência desnecessária dentro do perímetro de rede corporativo. Em vez de expor APIs e bancos de dados por meio de instâncias continuamente ativas, a arquitetura redefine o modelo de conectividade ao introduzir o conceito de Micro-MCPs efêmeros.

A hipótese central do Z-MESH é que os riscos associados ao uso de agentes de inteligência artificial não estão no protocolo MCP em si, mas no modelo operacional de sua implementação quando baseado em servidores persistentes, multifuncionais e com permissões agregadas. Assim, a segurança de agentes de IA pode ser aprimorada ao substituir esse modelo por uma abordagem baseada em capacidades efêmeras, isoladas, auditáveis e governadas por política.

No paradigma proposto, o acesso a ferramentas MCP deixa de estar associado a servidores persistentes e passa a ser tratado como execução temporária de capacidades específicas. Para cada solicitação do agente, o Z-MESH instancia dinamicamente um Micro-MCP especializado, definido para uma finalidade específica, executado em runtime de sandbox isolado, com credenciais temporárias de escopo mínimo e restritas ao contexto da requisição.

Essa mudança altera a superfície de risco ao reduzir conectores multifuncionais continuamente expostos, evitar o acúmulo de permissões em identidades técnicas permanentes e limitar o impacto de abusos ou comprometimentos a uma execução específica. Além disso, ao vincular cada chamada a um contexto

explícito de identidade, finalidade e política, o Z-MESH permite governança granular, auditoria contextual e controle determinístico sobre o uso de capacidades por agentes autônomos.

Dessa forma, o agente deixa de possuir acesso direto ou persistente a sistemas internos. Em vez disso, passa a interagir exclusivamente com capacidades efêmeras intermediadas, criadas sob demanda, executadas sob autorização e destruídas ao final da operação, transformando o modelo de integração contínua em um modelo de execução controlada e transitória.

A **Figura 4** apresenta o posicionamento conceitual do Z-MESH em relação às principais camadas já existentes no ecossistema de agentes de IA. Frameworks como Semantic Kernel, LangGraph e AutoGen atuam predominantemente na orquestração lógica de agentes, fluxos e chamadas de ferramentas. Plataformas de sandbox agentic, como E2B, Daytona e Browserbase, oferecem ambientes isolados para execução de código, ferramentas, arquivos e automações. O Z-MESH, por sua vez, posiciona-se como uma camada complementar de governança Zero Trust, responsável por controlar o ciclo de vida seguro dos Micro-MCPs efêmeros. Sua contribuição está em decidir quando uma capacidade pode ser acionada, qual Micro-MCP deve ser instanciado, em qual runtime de sandbox será executado, com quais credenciais temporárias, sob quais restrições de tráfego, com qual nível de auditoria e por quanto tempo esse ambiente poderá existir.

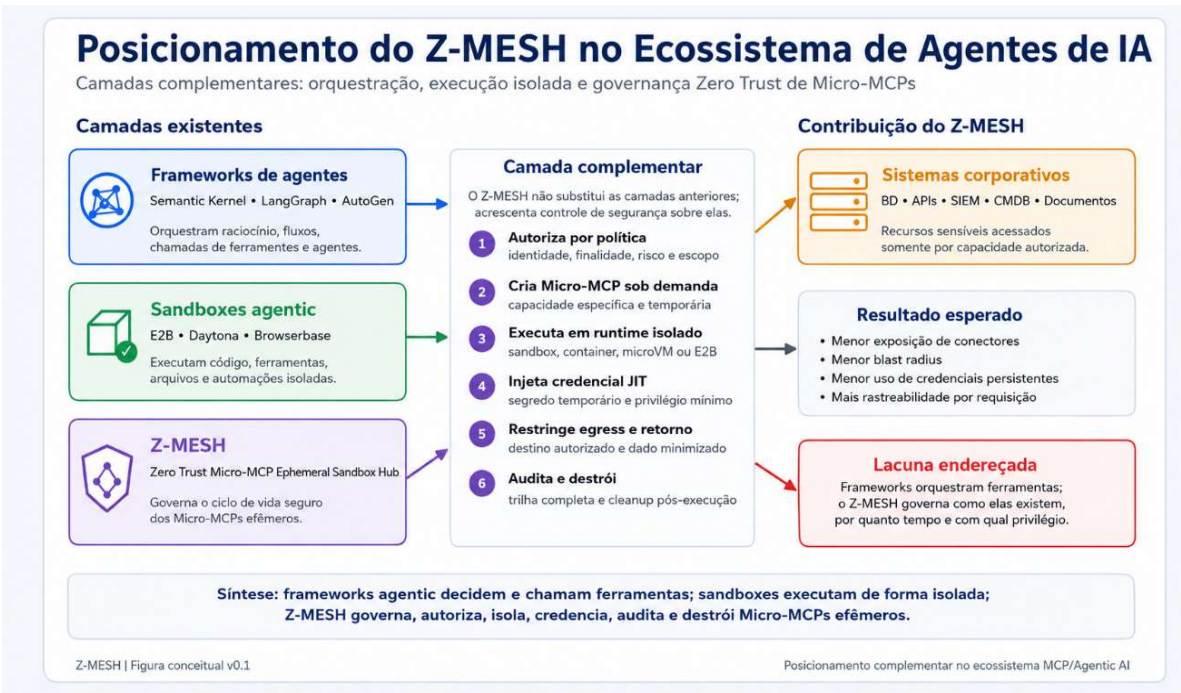


Figura 4 — Posicionamento do Z-MESH em relação a frameworks de agentes e plataformas de sandbox.

Assim, o agente não recebe acesso direto ao sistema interno. Ele recebe apenas uma resposta mediada por uma capacidade efêmera, criada para uma finalidade

específica, executada em ambiente isolado, auditada e destruída após a execução. A partir dessa premissa, a arquitetura Z-MESH pode ser descrita como um conjunto de camadas coordenadas, responsáveis por transformar solicitações de agentes de IA em execuções controladas de Micro-MCPs, com política, identidade, escopo mínimo, credenciais temporárias e destruição automática do runtime.

4. Visão geral da arquitetura Z-MESH

A arquitetura Z-MESH — Zero Trust Micro-MCP Ephemeral Sandbox Hub é proposta como um plano de controle seguro para mediar o acesso de agentes de inteligência artificial a sistemas corporativos por meio de Micro-MCPs efêmeros, executados em runtimes de sandbox isolados e governados por política.

Diferentemente do modelo tradicional, no qual servidores MCP podem permanecer ativos de forma contínua na rede corporativa, o Z-MESH adota uma abordagem baseada em execução transitória. Nesse modelo, o agente de IA não acessa diretamente bancos de dados, APIs, repositórios documentais ou serviços internos. Ele apenas envia uma solicitação de capacidade ao Concentrador Z-MESH. A partir dessa solicitação, a arquitetura avalia identidade, finalidade, política, escopo e risco antes de instanciar, de forma controlada, o Micro-MCP necessário para executar a tarefa autorizada.

A arquitetura é composta por cinco camadas principais:

1. Camada de agentes e orquestradores de IA.
2. Concentrador Z-MESH.
3. Motor de políticas, identidade e segredos.
4. Runtimes de sandbox distribuídos.
5. Micro-MCPs efêmeros e sistemas corporativos de destino.

Essas camadas atuam de forma coordenada para transformar uma intenção do agente em uma execução controlada, auditável e temporária de uma capacidade MCP específica.

4.1. Camada de agentes e orquestradores

A primeira camada representa os consumidores da arquitetura Z-MESH. Nela estão os agentes de IA, copilotos corporativos, frameworks agentic, pipelines DevSecAI e orquestradores como CrewAI, LangGraph, Semantic Kernel, AutoGen ou soluções equivalentes.

Esses agentes podem interpretar objetivos, planejar ações, selecionar ferramentas e solicitar dados ou operações a partir de sistemas internos. Contudo, no modelo Z-MESH, eles não recebem acesso direto a credenciais, endereços internos, bancos de dados, APIs ou repositórios corporativos.

O agente apenas envia uma solicitação de capacidade ao Concentrador Z-MESH, informando o que precisa realizar, para qual finalidade e sobre qual recurso.

Exemplo lógico de solicitação:

```
{  
  "agent_id": "agente-analise-contratos",  
  "user_id": "analista-001",  
  "purpose": "avaliar risco contratual",  
  "resource": "contratos",  
  "action": "read",  
  "target": "contrato_001"  
}
```

Essa abordagem impede que o agente se conecte diretamente aos sistemas internos e permite que cada requisição seja avaliada antes de qualquer acesso real ao ambiente corporativo.

4.2. Concentrador Z-MESH

O Concentrador Z-MESH é o componente central da arquitetura. Ele atua como broker, plano de controle, ponto de decisão operacional e orquestrador do ciclo de vida dos Micro-MCPs efêmeros.

Sua função não é apenas rotear chamadas para ferramentas, mas controlar se, como, onde, com qual escopo e por quanto tempo uma capacidade MCP poderá existir.

Entre suas principais responsabilidades estão:

- Receber solicitações de capacidade enviadas por agentes ou orquestradores de IA.
- Autenticar o agente solicitante e identificar o usuário, workload ou processo associado.
- Avaliar a finalidade declarada e o contexto da requisição.
- Consultar o motor de políticas.
- Selecionar o Micro-MCP adequado;
- Escolher o runtime de sandbox apropriado;
- Solicitar ou emitir credenciais temporárias;
- Instanciar o Micro-MCP sob demanda;
- Restringir o tráfego permitido;
- Receber o resultado da execução;
- Aplicar minimização, filtragem ou mascaramento de dados;
- Registrar trilhas de auditoria;
- Encerrar o Micro-MCP e destruir o ambiente efêmero;
- Devolver uma resposta consolidada e controlada ao agente.

Dessa forma, o Concentrador Z-MESH substitui o acesso direto do agente por um modelo mediado, contextual e governado por política.

4.3. Motor de políticas, identidade e segredos

O motor de políticas é responsável por avaliar se uma solicitação pode ser executada, negada ou autorizada com restrições. Ele funciona como a camada de decisão da arquitetura, aplicando regras de controle de acesso, contexto, finalidade e risco.

A decisão pode considerar atributos como:

- Identidade do agente;
- Identidade do usuário associado;
- Origem da chamada;
- Finalidade declarada;
- Classificação do dado solicitado;
- Criticidade do recurso;
- Tipo de ação pretendida;
- Risco da sessão;
- Horário ou condição operacional;
- Necessidade de aprovação humana;
- Exigência de mascaramento ou minimização de dados;
- Escopo máximo da credencial temporária;
- Runtime autorizado para execução.

Exemplo de decisão de política:

```
{
  "decision": "allow_with_redaction",
  "allowed_runtime": "sandbox-contratos",
  "allowed_micro_mcp": "mcp-contratos-readonly",
  "credential_scope": "contratos.readonly",
  "credential_ttl_seconds": 120,
  "redact_fields": ["cpf", "dados_bancarios"],
  "audit_required": true
}
```

Essa camada também pode se integrar a serviços de identidade, cofre de segredos, mecanismos de credenciais dinâmicas, trilhas de auditoria, soluções de SIEM/SOC e plataformas de governança corporativa.

O objetivo é garantir que cada execução ocorra sob o princípio de menor privilégio, com credenciais temporárias, finalidade explícita e escopo restrito.

4.4. Runtimes de sandbox distribuídos

Os runtimes de sandbox são os ambientes de execução isolados nos quais os Micro-MCPs são criados, executados e encerrados. Eles representam o plano de execução da arquitetura Z-MESH.

Em uma prova de conceito inicial, esses runtimes podem ser implementados com containers efêmeros. Em ambientes de maior criticidade, podem evoluir para mecanismos de isolamento mais robustos, como runtimes sandboxed, gVisor, Kata Containers, microVMs ou ambientes baseados em Firecracker.

Esses runtimes podem ser distribuídos estrategicamente pela rede corporativa, próximos aos domínios de dados ou serviços que precisam acessar. Por exemplo:

- Runtime de sandbox para dados bancários.
- Runtime de sandbox para contratos.
- Runtime de sandbox para APIs internas.
- Runtime de sandbox para serviços de segurança.
- Runtime de sandbox para ambiente cloud.
- Runtime de sandbox para dados de baixa criticidade.

Cada runtime deve operar com negação por padrão, rede restrita, ausência de comunicação lateral desnecessária e destruição automática após a execução da tarefa.

Em termos de segurança, essa camada reduz a exposição permanente de conectores e limita o raio de impacto de uma execução indevida ou comprometida, pois cada Micro-MCP existe apenas durante o tempo necessário para cumprir uma finalidade previamente autorizada.

4.4.1. Microsegmentação e controle de tráfego dos runtimes

A proposta Z-MESH também prevê que cada runtime de sandbox opere sob um modelo de negação por padrão, no qual nenhum tráfego é permitido sem autorização explícita. A comunicação dos Micro-MCPs deve ser limitada por políticas de ingress e egress associadas ao contexto da requisição, ao domínio de dados acessado e à finalidade autorizada pelo motor de políticas.

Nesse modelo, a segurança de rede não depende apenas do isolamento do container ou da microVM, mas também da aplicação de microsegmentação dinâmica de tráfego Leste-Oeste. Cada Micro-MCP deve possuir rotas e destinos permitidos de forma explícita, comunicando-se apenas com o sistema de destino necessário para a tarefa autorizada. A comunicação lateral entre Micro-MCPs deve ser bloqueada por desenho, evitando que uma execução comprometida utilize o runtime como ponto de pivô para outros conectores, domínios ou sistemas internos.

Dessa forma, o Z-MESH combina isolamento de execução com controle granular de rede. O runtime não apenas contém o Micro-MCP, mas também restringe seu

raio de comunicação, reduzindo a superfície de ataque, a possibilidade de movimentação lateral e o impacto de uma eventual execução indevida.

4.4.2. Mitigação do *cold start*

A utilização de Micro-MCPs efêmeros impõe um desafio operacional significativo no que se refere ao tempo de inicialização do runtime. Como a arquitetura preconiza a criação de recursos sob demanda e sua destruição após a execução, o fenômeno de *cold start* pode se tornar um fator crítico para a experiência do usuário e para a sustentabilidade de fluxos de baixa latência.

Para mitigar esse desafio, o Z-MESH pode adotar estratégias de pré-aquecimento controlado, como *warm-pooling* de runtimes limpos, snapshots de microVMs sem contexto sensível, imagens previamente assinadas e carregadas em cache, containers ultraleves ou runtimes baseados em WebAssembly para casos compatíveis. Nessas abordagens, o ambiente base pode permanecer preparado para execução, mas sem credenciais, sem dados sensíveis e sem contexto de requisição. A identidade, os segredos, as políticas de rede e os parâmetros da tarefa só devem ser aplicados após a autorização pelo Concentrador Z-MESH.

Essa separação permite equilibrar segurança e desempenho. O runtime pode ser inicializado de forma mais rápida, sem abandonar os princípios de efemeridade, privilégio mínimo e isolamento contextual. Ainda assim, mecanismos de *warm-pool* devem possuir expiração, limpeza de estado, verificação de integridade e destruição periódica, para evitar que a otimização de performance reintroduza persistência indesejada na arquitetura.

4.5. Micro-MCPs efêmeros

Os Micro-MCPs são servidores MCP mínimos, especializados e descartáveis. Cada Micro-MCP deve ser projetado para executar uma capacidade específica ou um conjunto muito restrito de ações correlatas.

Ao contrário de servidores MCP genéricos, amplos e continuamente ativos, os Micro-MCPs no Z-MESH são instanciados apenas sob demanda, dentro de um runtime de sandbox, com credenciais temporárias e escopo mínimo.

Exemplos de Micro-MCPs:

- mcp-contratos-readonly.
- mcp-consulta-saldo-readonly.
- mcp-cambio-query.
- mcp-cmdb-readonly.
- mcp-vulnerabilidades-readonly.
- mcp-siem-alertas-readonly.
- mcp-fornecedor-risk-query.

Um Micro-MCP não deve conter credenciais embutidas nem permissões permanentes. As credenciais devem ser emitidas dinamicamente pelo

Concentrador Z-MESH, por um cofre de segredos ou por um mecanismo de identidade de workload, sempre com tempo de vida curto e escopo mínimo.

Além disso, cada Micro-MCP deve possuir regras explícitas sobre:

- Quais ações pode executar.
- Quais destinos de rede pode acessar.
- Quais dados pode retornar.
- Quais campos devem ser mascarados.
- Qual o tempo máximo de execução.
- Qual trilha de auditoria deve ser registrada.
- Em quais condições deve ser encerrado.

Essa abordagem transforma o MCP de um conector persistente em uma capacidade transitória, autorizada, isolada e auditável.

4.5.1. Injeção Just-in-Time de credenciais

No modelo Z-MESH, os Micro-MCPs não devem conter credenciais embutidas, tokens permanentes ou segredos armazenados na imagem do runtime. As credenciais devem ser emitidas apenas após a autorização da requisição, com tempo de vida curto, escopo mínimo e vínculo explícito com a finalidade aprovada pelo motor de políticas.

A injeção de credenciais pode ocorrer no momento de *bootstrap* do runtime efêmero, por meio de mecanismos controlados pelo Concentrador Z-MESH, pelo orquestrador ou por um cofre de segredos integrado. Preferencialmente, os segredos devem ser disponibilizados em memória, por variáveis de ambiente de curta duração, arquivos temporários em volumes efêmeros, montagem dinâmica em *tmpfs* ou mecanismos equivalentes que evitem persistência em disco e reduzam a exposição após o encerramento da execução.

Essa abordagem não elimina a necessidade de proteger o próprio plano de controle. O Concentrador, o orquestrador, o cofre de segredos e o runtime devem ser endurecidos e auditados, pois o comprometimento dessas camadas pode expor o processo de emissão ou entrega das credenciais. Ainda assim, ao substituir segredos permanentes por credenciais temporárias, contextuais e descartáveis, o Z-MESH reduz significativamente a janela de exploração e o impacto de eventual vazamento.

4.6. Síntese da arquitetura

A arquitetura Z-MESH pode ser compreendida como uma separação entre plano de controle e plano de execução.

O plano de controle, representado pelo Concentrador Z-MESH, motor de políticas, identidade e segredos, decide se uma capacidade pode ser acionada, em qual contexto, com qual escopo e sob quais restrições.

O plano de execução, representado pelos runtimes de sandbox e pelos Micro-MCPs efêmeros, executa a ação autorizada de forma isolada, temporária e rastreável.

Dessa forma, o Z-MESH cria uma camada complementar ao ecossistema MCP, permitindo que agentes de IA consumam capacidades corporativas sem receber acesso direto, amplo ou persistente aos sistemas internos. O resultado é um modelo de integração baseado em *Zero Trust*, privilégio mínimo, efemeridade, isolamento contextual, credenciais temporárias e auditoria integral.

4.7. Por que o Z-MESH é uma arquitetura Zero Trust

O Z-MESH pode ser caracterizado como uma arquitetura *Zero Trust* porque elimina a confiança implícita entre agentes de IA, conectores MCP e sistemas corporativos. Em vez de permitir que um agente acesse diretamente servidores MCP persistentes ou sistemas internos previamente expostos na rede, cada solicitação passa a ser tratada como uma nova decisão de segurança.

Nesse modelo, nenhuma chamada é considerada confiável apenas porque parte de um agente autorizado, de uma aplicação conhecida ou de uma rede interna. Cada requisição precisa ser avaliada em função de identidade, finalidade, contexto, escopo, criticidade do recurso, política vigente e risco da operação. A autorização deixa de ser estática e passa a ser contextual, vinculada à tarefa específica que o agente pretende executar.

O Z-MESH também aplica o princípio de privilégio mínimo ao substituir conectores amplos por Micro-MCPs especializados. Cada Micro-MCP nasce apenas para executar uma capacidade limitada, com credenciais temporárias, acesso de rede restrito e tempo de vida reduzido. Após a conclusão da tarefa, o runtime é destruído, evitando a permanência de conectores, sessões, credenciais ou permissões ativas além do necessário.

Assim, o modelo Zero Trust do Z-MESH não se limita à autenticação do agente. Ele se manifesta em todo o ciclo de vida da execução: na validação da solicitação, na decisão de política, na emissão de credenciais, na escolha do runtime, na restrição de tráfego, na minimização da resposta, na auditoria contextual e na destruição automática do ambiente efêmero.

4.8. Vantagens arquiteturais do modelo Z-MESH

A principal vantagem do Z-MESH é reduzir a exposição contínua de servidores MCP persistentes na rede corporativa. Em arquiteturas tradicionais, cada novo conector MCP pode se tornar um ponto adicional de proteção, monitoramento, atualização, hardening, gestão de credenciais e auditoria. Em ambientes de grande escala, essa multiplicação de conectores aumenta a complexidade operacional e cria uma superfície de ataque distribuída.

O Z-MESH propõe uma abordagem diferente. Em vez de espalhar múltiplos servidores MCP permanentes pela rede, a arquitetura centraliza a governança das capacidades em um plano de controle Zero Trust. O agente não escolhe livremente quais conectores acessar. Ele solicita uma capacidade ao

Concentrador Z-MESH, que decide se a ação pode ser executada, qual Micro-MCP deve ser criado, em qual runtime de sandbox ele será instanciado, com qual credencial temporária, por quanto tempo e com quais restrições de tráfego e retorno de dados.

Essa mudança traz ganhos relevantes:

- Redução da superfície de ataque associada a conectores continuamente ativos.
- Diminuição do raio de impacto em caso de comprometimento de uma execução.
- Eliminação ou redução do uso de credenciais permanentes em conectores MCP.
- Aplicação mais rigorosa de privilégio mínimo.
- Isolamento entre capacidades distintas.
- Menor risco de movimentação lateral.
- Auditoria contextual por requisição, agente, finalidade e runtime.
- Maior controle sobre quais dados podem retornar ao agente.
- Simplificação da governança de segurança sobre ferramentas MCP.
- Possibilidade de distribuição controlada dos runtimes próximos aos domínios de dados.

O resultado é uma arquitetura em que a integração deixa de depender de conectores estáticos e passa a operar por meio de capacidades transitórias, governadas e descartáveis.

5. Da sandbox isolada ao hub de governança de Micro-MCPs

O conceito tradicional de sandbox está associado à contenção de uma execução potencialmente arriscada. Em soluções clássicas de segurança, a sandbox é usada para executar arquivos, scripts ou comportamentos suspeitos em um ambiente isolado, reduzindo o impacto sobre o sistema principal. No contexto de agentes de IA e MCP, o Z-MESH amplia esse conceito.

A proposta não é apenas executar um Micro-MCP dentro de uma sandbox. A contribuição arquitetural está em transformar o uso de runtimes isolados em uma malha governada de capacidades MCP efêmeras. Nesse modelo, o Concentrador Z-MESH atua como ponto lógico central de decisão e governança, enquanto os runtimes de sandbox funcionam como ambientes distribuídos de execução controlada.

Essa distinção é importante. O Z-MESH não precisa ser um único equipamento físico nem um ponto único de falha. Ele pode ser logicamente centralizado e fisicamente distribuído, com múltiplos runtimes posicionados próximos aos sistemas de destino. O que se centraliza é a política, a identidade, a decisão, a auditoria e o controle do ciclo de vida dos Micro-MCPs. O que se distribui é a execução isolada, conforme a localização, criticidade e domínio dos dados.

Dessa forma, o Z-MESH transforma o sandboxing de um mecanismo passivo de contenção em um plano ativo de governança de capacidades. Em vez de manter servidores MCP espalhados e permanentemente ativos pela rede, a arquitetura

cria Micro-MCPs sob demanda, controla sua existência, limita seus privilégios, registra sua atuação e encerra o ambiente ao final da tarefa.

Essa abordagem reforça o objetivo central da proposta: permitir que agentes de IA consumam capacidades corporativas sem receber acesso direto, amplo ou persistente aos sistemas internos.

6. Proposta de validação experimental

Embora a arquitetura Z-MESH ainda não tenha sido validada experimentalmente nesta versão do trabalho, propõe-se um modelo inicial de laboratório para demonstrar sua viabilidade funcional em ambiente controlado. O objetivo dessa validação futura é comprovar o fluxo lógico da arquitetura, especialmente a mediação do acesso pelo Concentrador Z-MESH, a criação sob demanda de Micro-MCPs efêmeros, a execução em ambiente isolado, o retorno controlado da resposta e a destruição automática do runtime após a conclusão da tarefa.

A prova de conceito inicial poderá ser construída em ambiente local utilizando tecnologias de baixo custo e fácil reprodução, como Docker, Python e FastAPI. Nessa primeira etapa, os containers efêmeros funcionarão como runtimes isolados para fins de demonstração funcional, sem a pretensão de representar o nível de isolamento exigido para ambientes produtivos de alta criticidade.

O laboratório proposto teria os seguintes componentes mínimos:

- Agente simulador em Python.
- Concentrador Z-MESH implementado em FastAPI.
- Dois ou três Micro-MCPs empacotados em containers Docker.
- APIs mockadas representando sistemas internos.
- Regras simples de autorização.
- Logs de auditoria em arquivo JSON.

O fluxo de validação consistiria em submeter uma requisição do agente ao Concentrador Z-MESH, avaliar a política aplicável, instanciar o Micro-MCP correspondente em container efêmero, executar a ação autorizada contra uma API simulada, retornar o resultado ao Concentrador, registrar a trilha de auditoria e remover automaticamente o runtime ao final da execução.

A PoC será considerada bem-sucedida se demonstrar que o agente não acessa diretamente o sistema mockado, que o Micro-MCP é criado apenas sob demanda, que a execução ocorre com escopo limitado, que requisições não autorizadas são bloqueadas antes da criação do runtime, que cada execução gera trilha de auditoria e que o ambiente efêmero é destruído após o uso.

Essa proposta de laboratório não tem como objetivo comprovar, nesta fase, segurança produtiva em nível bancário ou isolamento forte equivalente a microVMs. Seu objetivo inicial é validar a lógica funcional do Z-MESH. Em etapas posteriores, a arquitetura poderá ser evoluída para runtimes mais robustos, como gVisor, Kata Containers, Firecracker ou plataformas de sandbox agentic,

além da integração com motores de política, cofres de segredo, identidade de workload e observabilidade corporativa.

7. Limitações da proposta

Por se tratar de um modelo conceitual inicial, a arquitetura Z-MESH apresenta limitações e desafios que deverão ser avaliados em etapas futuras de experimentação. Entre os principais pontos a serem analisados estão o impacto de latência associado à criação de runtimes efêmeros, a complexidade operacional do plano de controle, o custo de infraestrutura, a necessidade de mecanismos robustos de proteção do Concentrador Z-MESH, a segurança do processo de emissão de credenciais temporárias e a efetividade dos controles de isolamento em diferentes tecnologias de runtime.

Essas limitações não invalidam a proposta, mas delimitam seu escopo nesta versão inicial e indicam pontos relevantes para evolução, validação prática e amadurecimento arquitetural. Dessa forma, o Z-MESH deve ser compreendido como uma proposta conceitual em desenvolvimento, cuja viabilidade funcional e segurança operacional deverão ser demonstradas progressivamente por meio de provas de conceito, experimentação controlada e avaliação em cenários corporativos simulados.

8. Trabalhos futuros e expansão para arquiteturas AI-WAF

Como perspectiva de evolução, o Z-MESH pode ser expandido para atuar como componente de uma arquitetura mais ampla de proteção para aplicações baseadas em inteligência artificial, aproximando-se do conceito de Next-Gen AI-WAF. Nesse cenário, a proposta deixaria de se limitar à governança de Micro-MCPs efêmeros e passaria a compor uma camada integrada de defesa para tráfego tradicional, tráfego de modelos de linguagem e ações executadas por agentes autônomos.

Em uma arquitetura convencional, um WAF protege aplicações web contra ataques conhecidos, abuso de requisições HTTP, exploração de vulnerabilidades e padrões anômalos de tráfego. Já em aplicações baseadas em IA generativa, a superfície de ataque se amplia para elementos semânticos, como prompt injection, jailbreak, vazamento de dados, manipulação de contexto, uso indevido de ferramentas e acionamento não autorizado de capacidades externas. Nesse contexto, um AI-WAF não deve apenas inspecionar pacotes, URLs ou parâmetros de requisição, mas também avaliar intenção, contexto, finalidade, risco da ação solicitada e impacto potencial da execução.

O Z-MESH poderia contribuir para essa evolução ao atuar na fronteira entre a decisão do agente e a execução real sobre sistemas corporativos. Enquanto camadas de AI Gateway, LLM Firewall ou DLP podem analisar prompts, respostas e dados sensíveis, o Z-MESH adiciona uma camada de controle sobre a ação resultante. Quando uma interação com IA exigir acesso a um sistema interno, a arquitetura avaliaria a política, instanciaria um Micro-MCP efêmero em runtime de sandbox, emitiria credenciais temporárias, restringiria o tráfego permitido, minimizaria a resposta e destruiria o ambiente após a execução.

Dessa forma, o Z-MESH poderia evoluir para uma função complementar ao WAF tradicional e às soluções de segurança específicas para IA. Sua contribuição estaria em proteger não apenas a entrada e a saída textual do modelo, mas também o momento mais crítico da cadeia agentic: a transformação de uma intenção semântica em uma ação concreta sobre APIs, bancos de dados, documentos, ferramentas operacionais ou sistemas internos.

Essa evolução posicionaria o Z-MESH como uma camada de execução segura dentro de uma arquitetura de defesa mais ampla, capaz de combinar inspeção tradicional de tráfego, análise semântica de interações com LLMs, governança de ferramentas, execução efêmera, credenciais temporárias e auditoria contextual. Assim, o modelo poderia contribuir para uma nova geração de controles de segurança voltados a ambientes nos quais aplicações web, agentes de IA e sistemas corporativos passam a operar de forma cada vez mais integrada.

9. Contribuições esperadas

A principal contribuição esperada do Z-MESH é propor uma arquitetura complementar ao ecossistema MCP, voltada à redução dos riscos associados à exposição contínua de servidores MCP persistentes em ambientes corporativos críticos. A proposta não busca substituir o Model Context Protocol, frameworks agentic ou plataformas de sandbox, mas adicionar uma camada de governança Zero Trust para controlar o ciclo de vida das capacidades acionadas por agentes de IA.

Nesse sentido, o Z-MESH contribui ao deslocar o modelo de integração de conectores permanentes para capacidades efêmeras, especializadas e auditáveis. Em vez de manter servidores MCP amplos, continuamente ativos e potencialmente associados a credenciais persistentes, a arquitetura propõe a criação sob demanda de Micro-MCPs com escopo mínimo, executados em runtimes de sandbox isolados, com credenciais temporárias, egresso restrito e destruição automática após a execução.

Outra contribuição relevante é a separação clara entre orquestração lógica do agente e governança segura da execução. Frameworks como Semantic Kernel, LangGraph e AutoGen podem continuar atuando na coordenação de agentes, fluxos e chamadas de ferramentas, enquanto o Z-MESH atua como plano de controle de segurança, avaliando política, identidade, finalidade, escopo e risco antes de permitir que uma capacidade seja executada sobre sistemas corporativos sensíveis.

A proposta também contribui ao introduzir o conceito de Micro-MCPs efêmeros como unidade mínima de capacidade segura. Essa abordagem favorece a aplicação de privilégio mínimo, reduz o raio de impacto em caso de comprometimento, limita a movimentação lateral, melhora a rastreabilidade por requisição e permite maior controle sobre quais dados podem retornar ao agente.

Além disso, o Z-MESH oferece uma base conceitual para futuras validações experimentais, permitindo que a arquitetura seja testada inicialmente em

laboratório com containers efêmeros e, posteriormente, evoluída para runtimes mais robustos, como microVMs, gVisor, Kata Containers, Firecracker ou plataformas de sandbox agentic. Dessa forma, a proposta estabelece um caminho progressivo entre formulação conceitual, prova de viabilidade funcional e eventual aplicação em ambientes corporativos de maior criticidade.

Em síntese, as contribuições esperadas do Z-MESH são:

- Propor uma arquitetura Zero Trust específica para governança de capacidades MCP acionadas por agentes de IA.
- Reduzir a dependência de servidores MCP persistentes, amplos e continuamente expostos.
- Introduzir Micro-MCPs efêmeros como unidade de execução segura e especializada.
- Aplicar credenciais temporárias, escopo mínimo, egresso restrito e destruição pós-execução ao ciclo de vida das ferramentas.
- Separar a orquestração lógica dos agentes da governança segura da execução sobre sistemas internos.
- Melhorar auditoria, rastreabilidade e controle contextual das ações realizadas por agentes autônomos.
- Oferecer uma base conceitual para validação experimental futura e evolução para arquiteturas de segurança mais amplas, incluindo modelos de AI-WAF e proteção de tráfego agentic.

10. Conclusão

A adoção de agentes de inteligência artificial em ambientes corporativos tende a ampliar a automação, a integração entre sistemas e a capacidade de execução de tarefas complexas. Entretanto, essa evolução também desloca parte significativa do risco para a camada de integração entre agentes, ferramentas, APIs, bancos de dados, repositórios documentais e serviços internos. Nesse contexto, o Model Context Protocol representa um avanço relevante ao padronizar a comunicação entre aplicações de IA e sistemas externos, reduzindo a complexidade de integrações ponto a ponto e favorecendo interoperabilidade, reutilização e escalabilidade.

O problema central identificado neste trabalho não está no MCP como protocolo, mas no modelo operacional de implantação baseado em servidores MCP persistentes, amplos e continuamente expostos na rede corporativa. Quando esses conectores permanecem ativos de forma permanente, acumulam múltiplas capacidades, utilizam credenciais de longa duração ou operam com escopos excessivos, podem reproduzir riscos já conhecidos em APIs, integrações legadas e contas técnicas, como permissões excessivas, dificuldade de auditoria, movimentação lateral, ampliação do raio de impacto e acionamento indevido de ferramentas por agentes comprometidos ou induzidos por prompt injection.

Diante desse cenário, o Z-MESH propõe uma abordagem arquitetural complementar ao ecossistema MCP. Sua contribuição está em substituir o modelo de conectores persistentes por capacidades efêmeras, especializadas, isoladas e governadas por política. No modelo proposto, o agente de IA não acessa diretamente o sistema interno nem se conecta livremente a servidores MCP distribuídos pela rede. Ele solicita uma capacidade ao Concentrador Z-MESH, que avalia identidade, finalidade, contexto, política, risco e escopo antes de instanciar o Micro-MCP necessário em um runtime de sandbox isolado.

Essa mudança permite aplicar, de forma integrada, princípios de Zero Trust, privilégio mínimo, efemeridade, isolamento contextual, credenciais temporárias, controle de egress, minimização de dados e auditoria integral. O acesso deixa de ser contínuo e passa a ser transitório. O conector deixa de ser uma presença permanente na rede e passa a existir apenas durante a execução de uma tarefa autorizada. A credencial deixa de ser um segredo persistente associado a um servidor amplo e passa a ser emitida sob demanda, com tempo de vida curto e escopo limitado.

A proposta também reforça a separação entre orquestração lógica de agentes e governança segura da execução. Frameworks como Semantic Kernel, LangGraph, AutoGen e plataformas de sandbox para agentes podem continuar desempenhando papéis relevantes na construção, coordenação e execução de workflows agentic. O Z-MESH não substitui essas camadas. Ele atua como plano de controle complementar, voltado à decisão, autorização, isolamento, credenciamento, auditoria e destruição das capacidades MCP acionadas por agentes de IA.

Por se tratar de uma versão inicial, este documento apresenta o Z-MESH como proposta conceitual e arquitetural, ainda pendente de validação experimental. O modelo de laboratório proposto com containers efêmeros, APIs mockadas, políticas simples e trilhas de auditoria busca estabelecer um caminho inicial para demonstrar a viabilidade funcional da arquitetura. Em etapas futuras, a proposta poderá ser evoluída para runtimes mais robustos, integração com motores de política, cofres de segredo, identidade de workload, observabilidade corporativa e mecanismos de isolamento mais fortes, como microVMs ou runtimes sandboxed.

Em síntese, o Z-MESH busca contribuir para uma adoção mais segura, controlada e auditável da inteligência artificial agentic em ambientes corporativos sensíveis. Sua premissa central é que agentes de IA não devem receber acesso direto, amplo ou persistente a sistemas internos. Em vez disso, devem interagir com capacidades efêmeras, autorizadas, isoladas e auditáveis, criadas sob demanda e destruídas ao final da execução. Essa abordagem estabelece uma base conceitual para futuras validações experimentais e para a evolução de arquiteturas Zero Trust aplicadas à integração entre agentes de IA, ferramentas e sistemas corporativos.

11. Referências

- [1] ANTHROPIC. Introducing the Model Context Protocol. Anthropic, 2024.
- [2] MODEL CONTEXT PROTOCOL. Model Context Protocol Documentation and Specification. Model Context Protocol Project.
- [3] OPENAI. MCP and Connectors: Tools and Connectors Guide. OpenAI Developers Documentation.
- [4] GOOGLE CLOUD. What is Model Context Protocol? Google Cloud Documentation.
- [5] MICROSOFT. Model Context Protocol overview. Microsoft Learn.
- [6] MICROSOFT. Semantic Kernel Documentation. Microsoft Learn.
- [7] MICROSOFT. Give agents access to MCP Servers. Semantic Kernel Documentation. Microsoft Learn.
- [8] MICROSOFT. Magentic-One: A generalist multi-agent system for solving complex tasks. Microsoft Research / AutoGen Documentation.
- [9] LANGCHAIN. Model Context Protocol integration. LangChain Documentation.
- [10] LANGGRAPH. LangGraph Documentation: agents, graphs and orchestration. LangChain Documentation.
- [11] E2B. E2B Documentation: sandboxes for AI agents. E2B Documentation.
- [12] DOCKER. Docker Compose Documentation. Docker Documentation.
- [13] FIRECRACKER MICROVM. Firecracker Documentation and Project Repository. Firecracker MicroVM Project.
- [14] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. NIST SP 800-207: Zero Trust Architecture. NIST, 2020.
- [15] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. NIST SP 800-53: Security and Privacy Controls for Information Systems and Organizations. NIST.
- [16] OWASP FOUNDATION. OWASP API Security Top 10. OWASP.
- [17] OWASP FOUNDATION. OWASP Top 10 for Large Language Model Applications. OWASP GenAI Security Project.
- [18] OWASP FOUNDATION. MCP Security Cheat Sheet. OWASP Cheat Sheet Series.
- [19] OPEN POLICY AGENT. Open Policy Agent Documentation. OPA Project.
- [20] HASHICORP. Vault Documentation: static and dynamic secrets. HashiCorp Developer Documentation.

[21] SPIFFE / SPIRE. Secure Production Identity Framework for Everyone. SPIFFE Project.

[22] HOHPE, Gregor; WOOLF, Bobby. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Boston: Addison-Wesley, 2003.